

Workforce / Hospitable Updated Master Plan

Unified architecture, operating model, and phased implementation guide compiled from the reviewed archive.

Prepared: March 20, 2026

Scope: Workforce core, HKOps / Hospitable operations, Silver Sands integration, web console, event system, and AI widget builder.

Editorial stance: This document reconciles overlapping plans, removes duplicate exports, and resolves conflicts where later documents supersede earlier exploratory notes.

What changed in this consolidated edition

The archive contains parallel drafts, exported PDFs, duplicate markdown files, and a few design tensions. This edition normalizes them into one current direction: a single multi-tenant Workforce platform, with Hospitable as a first-class operational module, a Postgres-first event layer, and a unified web console.

Contents

- 1. Executive summary
- 2. Source corpus reviewed
- 3. Consolidated product position
- 4. Core architecture and platform planes
- 5. Multi-tenancy, scoping, and RBAC
- 6. Hospitable / HKOps operational domain
- 7. Silver Sands Motel implementation model
- 8. Workforce web console and workspace design
- 9. Event system and timeline architecture
- 10. AI widget agent
- 11. Data imports, staffing views, and operational screens
- 12. Delivery roadmap and first PR slices
- 13. Guardrails, risks, and normalization decisions
- 14. Recommended immediate deliverables

1. Executive summary

The reviewed archive points to one consistent target product even though the files were written at different times and at different levels of detail. The platform is a unified Workforce system that supports multi-business tenancy, location-scoped RBAC, workforce operations, and hospitality operations inside one backend and one web application.

Hospitable should not be treated as a separate app. It is the operations layer for property structure, room lifecycle, housekeeping tasks, inspections, issues, room assets, and supply standards, while Workforce remains the system of record for employees, schedules, timeclock, and labor data.

Silver Sands Motel is the first concrete property implementation. Its blueprint becomes a seeded location model inside Workforce: Business -> Location -> Building -> Floor -> Sector -> Room, with normalized room, task, issue, asset, and supply entities.

The event architecture should be adopted, but with a clear Phase 1 constraint: use a transactional Postgres-first event store and timeline API before introducing Kafka or other brokers. This preserves auditability and keeps delivery realistic.

The frontend direction is also clear after normalization: evolve the existing web UI into a shared Workforce console with workspaces for Superadmin, Tenant Admin, Workforce, and Hospitable. Re-home current Hospitable pages under /hospitable rather than starting from scratch.

The practical result is a phased build strategy that produces immediate operational value: tenant-aware HKOps data models, Silver Sands seeders, room board and dashboard APIs, bulk room actions, UI shell consolidation, and a disciplined event layer that later supports CRM timelines, AI-built widgets, and richer projections.

Area	Current consolidated decision
Product boundary	Single Workforce platform; Hospitable is a module/workspace, not a separate product.
Data authority	Workforce owns employees, schedules, timeclock, tenancy, RBAC; HKOps owns property operations and housekeeping execution.
First concrete property	Silver Sands Motel with normalized property hierarchy and seeded rooms 1–12.

Frontend direction	Extend the existing web app into a shared Workforce console with role-aware navigation and workspaces.
Event direction	Use an append-only Postgres event store and timeline API in Phase 1; add broker-based projection/publishing later if needed.
AI builder	Introduce an AI widget agent behind manifest validation, permission validation, and preview/publish workflow.

2. Source corpus reviewed

The folder included many duplicate exports and generated bundles. The following source documents were treated as the primary planning inputs for this master plan:

Category	Primary source documents used
Core platform	workforce_master_plan.md; REFACTORED_MASTER_PLAN.md
Silver Sands / HKOps	silver_sands_operations_blueprint.md; SILVER_SANDS_INTEGRATION_PLAN.md; HOSPITABLE_WORKFORCE_COPILOT_IMPLEMENTATION.md
UI / console	WORKFORCE_WEB_UI_PLAN.md; WORKFORCE_WEB_UI_COPILOT_IMPLEMENTATION.md; workforce_api_manus_frontend_brief.md
Event layer	EVENT_SYSTEM_BUILD_SPEC.md; Projection_Worker_System_Summary.md
AI widget layer	AI_WIDGET_AGENT_IMPLEMENTATION_PLAN.md
Operational screen specs	HKOPS_DASHBOARD_SPEC.md; HKOPS_TASKS_SPEC.md; HKOPS_INSPECTIONS_SPEC.md; HKOPS_ISSUES_SPEC.md; HKOPS_IMPORTS_SPEC.md; HKOPS_STAFF_SHIFTS_SPEC.md

Duplicate PDFs, zips, and repeated exports were used only as secondary confirmation when needed. Where files conflicted, the more implementation-ready and more recent direction was carried forward.

3. Consolidated product position

3.1 Platform identity

The archive repeatedly describes the same strategic model: a cloud-based Workforce platform with multiple logical planes operating on one backend. Those planes are control-plane administration, tenant/business administration, worker self-service, and agent/integration access.

Hospitable / HKOps belongs inside that platform as an operational module focused on property operations. It should not duplicate scheduling, payroll, or timeclock capabilities, and it should not ship as a disconnected hospitality-only stack.

3.2 Business value proposition

- Single source of truth for tenancy, business, location, user, and permission context.
- Operational visibility across rooms, tasks, inspections, issues, and staffing pressure.
- A direct path from prototype property operations to production-grade multi-location deployment.
- A future-ready platform that can absorb CRM timelines, AI-generated widgets, and cross-module event projections without rewriting the core.

3.3 Normalized product statement

Normalized statement

Build Workforce as the primary multi-tenant platform. Add Hospitable as the property-operations workspace for hospitality teams, seeded first with Silver Sands Motel, and support all modules through shared tenancy, RBAC, eventing, and web-console infrastructure.

4. Core architecture and platform planes

The core architecture stabilizes around one backend, one identity/RBAC system, one web console, and modular domain slices. Different user populations experience different workspaces, but the same platform primitives govern them all.

Plane	Primary responsibility	Examples
Control plane	Cross-tenant and system administration	Business provisioning, permission templates, feature flags, audit logs, integration and agent registry
Tenant plane	Business and location administration	Users, role assignments, locations, module enablement, reports, business settings
Worker plane	Day-to-day employee actions	My schedule, timeclock, assigned work, profile, requests
Agent plane	Scoped machine access	API keys, manifest-driven widget generation, integration runs, least-privilege automation
Hospitable workspace	Property operations and	Rooms, task queues,

	housekeeping execution	inspections, issues, supply pars, dispatch boards
--	------------------------	---

4.1 Shared architecture principles

- One backend, different scopes. UI visibility follows backend authorization, not the other way around.
- Transactional domain tables remain the source of truth; events complement them for audit, timeline, and projection use cases.
- Location scoping is first-class. Operational data must never leak across tenant or location boundaries.
- Prototype UX may start with mock or imported data, but production state must end up API-backed and auditable.

4.2 Recommended technology baseline

- FastAPI backend with SQLAlchemy and Alembic patterns already present in the repository family.
- PostgreSQL as the primary production database.
- Pydantic v2 for schemas and API contracts.
- Next.js web console with shared shell, typed client generation from OpenAPI where practical, and permission-aware navigation.
- Event storage in Postgres first, with publisher/projection workers introduced later as a controlled extension.

5. Multi-tenancy, scoping, and RBAC

The clearest and most mature RBAC model in the archive is location-aware and role-based. Permissions come from roles only. Users may hold different roles at different locations, while optional job-title labels remain presentation-only.

5.1 Core entities

Entity	Role in the model
Business	Tenant-level organizational boundary and settings context.
Location	Operational boundary for permissions, tasks, property assets, and reports.
Role	Named permission bundle; either business-scoped or location-scoped.
Permission	Atomic capability such as hospitable.rooms.write or platform.user.read.
User role assignment	The critical join that binds a user to a role at business or location scope, optionally with a display-only job title label.

5.2 Authorization rules

A user can act within a location if they hold a business-scoped role that includes the permission, or a location-scoped role at that exact location that includes the permission.

Job titles must never be used in permission checks. They are display labels attached to assignments so the UI can show a human-friendly title without hard-coding security logic into titles.

5.3 Delegated management

- Location owners or similarly privileged location-scoped roles may manage location-specific roles and assignments for their own locations.
- Business-wide role creation and cross-location role changes remain restricted unless explicitly granted.
- Audit logging is mandatory for role and assignment changes.

6. Hospitable / HKOps operational domain

Across the HKOps documents, the same domain emerges repeatedly: room-centric operations, housekeeping execution, quality control, issue handling, supply expectations, and dispatch visibility. The implementation should extend existing HKOps structures where they already exist rather than creating a parallel housekeeping subsystem.

6.1 Core operational entities

Entity group	Purpose
Rooms / units	Physical spaces that carry housekeeping, occupancy, inspection, and maintenance context.
Housekeeping tasks	Operational work queue: checkout clean, stayover, deep clean, restock, follow-up, and related task types.
Inspections	Quality-control records that determine whether completed room work passes release criteria.
Issues / maintenance flags	Room-linked defects, shortages, damage, safety concerns, and future maintenance escalations.
Room assets	Installed or expected equipment such as TVs, lamps, fridges, HVAC units, and furniture.
Supply pars	Expected room stock levels for linens, bath items, amenities, and consumables.
Operational events	Task transitions, room status history, audit logs, and timeline-compatible facts.

6.2 State model

- Room flow: dirty -> assigned -> cleaning / in progress -> clean -> inspect -> inspected / ready, with blocked or out-of-order branches.
- Task flow: open -> assigned -> in progress -> completed -> verified or inspected, with blocked and cancelled paths when necessary.
- Issue flow: open -> triaged / acknowledged -> in progress -> resolved -> closed.
- Inspection flow: pending -> pass / fail / pass with notes, with direct linkage to follow-up work when required.

6.3 Operational UX themes

- Queue-first design rather than generic CRUD pages.
- Fast inline status changes, assignment actions, and drill-down drawers instead of excessive page hopping.
- Supervisor-oriented dashboards that answer what needs attention right now, not just historical analytics.
- Clear distinction between housekeeping state, occupancy state, maintenance state, and inspection state.

7. Silver Sands Motel implementation model

Silver Sands is the first real property model in the archive and should be used as the seed case that validates the integrated architecture. Its current blueprint is simple enough to seed quickly while still exercising the full normalized hierarchy.

7.1 Property hierarchy

Level	Seeded structure for Silver Sands
Business	Silver Sands Motel
Location	Main Silver Sands property
Building	Building 1
Floor	Floor 1
Sectors	North Side; South Side
Room groups	North Group (7-12); South Group (1-6)
Rooms	Rooms 1 through 12 with future-safe string room identifiers

7.2 Data model implications

The motel blueprint maps cleanly to normalized `property_buildings`, `property_floors`, `property_sectors`, `hk_rooms`, `room_assets`, `room_supply_pars`, `hk_tasks`, `hk_task_events`, `maintenance_issues`, and `room status history` tables.

Room numbers should be stored as strings, not integers, so future identifiers such as 101A or ADA-1 do not force a migration.

Room setup should support bed metadata, floor surface, bathroom count, inspection requirements, room groups, and notes because these factors affect dispatch logic and quality standards.

7.3 Recommended first operational seed payload

- Create Building 1, Floor 1, North sector, and South sector.
- Seed rooms 1-6 in the South sector and rooms 7-12 in the North sector.
- Attach starter asset templates: TV, lamp, fridge, microwave, coffee maker, HVAC where relevant.
- Attach starter supply pars: towels, hand towels, washcloths, sheets, pillowcases, blankets, soap, shampoo, trash bags, and related consumables.
- Seed a minimal set of sample tasks, inspections, and issues so the room board and dashboards are operational immediately.

8. Workforce web console and workspace design

The UI documents all point toward the same practical conclusion: keep the existing web app, refactor it into a shared app shell, and separate work by workspace and permission scope rather than by creating multiple disconnected frontends.

8.1 Normalized route structure

Workspace	Representative routes
Root / overview	/ -> Workforce overview dashboard
Superadmin	/superadmin, /superadmin/businesses, /superadmin/users, /superadmin/roles, /superadmin/audit-log
Tenant Admin	/admin, /admin/locations, /admin/members, /admin/roles, /admin/business-settings
Workforce	/workforce, /workforce/scheduling, /workforce/shifts, /workforce/timeclock
Hospitable	/hospitable, /hospitable/rooms, /hospitable/housekeeping, /hospitable/maintenance, /hospitable/inventory, /hospitable/property-setup
System	/system/profile, /system/preferences, /system/access

8.2 UI shell requirements

- One shared AppShell with topbar, sidebar, content frame, mobile overlay handling, and context selectors.
- Permission-driven navigation registry rather than hard-coded menus inside components.
- Business selector and location selector as first-class context, never raw IDs shown as user-facing labels.
- Collapsible desktop sidebar, drawer-style mobile sidebar, and persistent theme preference.
- Backward-compatible redirects from old Hospitable root routes while pages are re-homed under /hospitable.

8.3 HKOps screen suite

Screen	Primary operational question
Dashboard	What needs attention right now?
Tasks	What work exists, who owns it, what is blocked, and what is overdue?
Inspections	Which rooms need QA review and what follow-up work is required?
Issues	What is wrong, how urgent is it, and which rooms

	are affected?
Staff Shifts	Who is working, how is labor distributed, and where are the gaps?
Imports / Data Sync	How can spreadsheet-managed operational data be validated and synchronized safely?
Room Board	What is the live state of rooms by room, sector, or group?

8.4 API and session alignment

The frontend should consume a normalized session/bootstrap payload that includes current user context, active business and location, memberships, role/permission lists, and enabled modules.

Where the live API already exists, generate or at least align client types from the OpenAPI schema. Business switching must refresh the token or active context and invalidate scoped caches.

9. Event system and timeline architecture

The event-system documents are strategically strong, but they require one normalization decision for execution: Phase 1 should be Postgres-first, transactional, and append-only. The projection-worker note that mentions Kafka should be treated as a later-scale option, not the starting dependency.

Normalization decision

Adopt the Event System Build Specification as the source of truth for Phase 1. Keep Kafka-style projection workers, DLQS, or external publishers as Phase 2+ extensions once the transactional event store and timeline API are stable.

9.1 Event envelope

- Every event needs a stable event_id, event_type, schema_version, tenant scope, aggregate reference, occurred_at, recorded_at, actor metadata, source module, payload, and related entity references.
- Events do not replace transactional tables. They record immutable facts about domain changes and support audit, timeline, observability, and later integration use cases.
- Related entities are essential so timelines can be rendered by room, task, issue, contact, deal, booking, and other objects without expensive ad hoc joins.

9.2 Phase 1 event catalog

Domain	Representative initial events
Operations	task.created, task.updated, task.completed, booking.created
Housekeeping / QA	inspection.completed, room-status related events
Maintenance	issue.created and follow-on issue state changes
CRM	contact.created, organization.created,

	deal.created, deal.stage_changed
Workforce	shift.started, shift.ended
Inventory	inventory.adjusted

9.3 Timeline strategy

Start with direct timeline reads from event_store plus either JSONB related-entity queries or a companion event_related_entity table for faster lookup.

Introduce a materialized timeline projection only when read complexity or query cost justifies it.

Write domain changes and events in the same transaction. If a domain mutation fails, the event must not persist; if the event write fails, the mutation must roll back.

9.4 Security and privacy

- Timeline visibility is governed by tenant, business, location, module permission, and entity-specific access rules.
- Do not store secrets or unnecessary sensitive PII in event payloads. Prefer stable IDs plus small business-relevant snapshots.
- Keep event types registry-driven to avoid uncontrolled sprawl or inconsistent payload contracts.

10. AI widget agent

The AI widget plan fits the platform well if it remains manifest-driven and validation-heavy. The agent should help generate dashboard widgets, not bypass layout, security, or backend governance.

Phase	Outcome
Foundation rules	Define manifest schema, approved primitives, discoverable permissions, and validation gates.
Persistence	Store widget definitions, permissions, instances, dashboard layouts, and agent runs.
Frontend registry	Render widgets through a controlled widgetRegistry and approved primitive components.
Agent draft endpoint	Generate draft manifests from prompts with validation, warnings, and dependency checks.
Preview and refine	Allow user-facing preview, editing, and re-validation before publication.
Publish and install	Turn approved drafts into reusable definitions and installed dashboard instances.
Domain packs	Improve drafting quality with housekeeping, maintenance, scheduling, inventory, and labor packs.

Backend stub support	Propose missing backend bindings as reviewable scaffolds, never auto-publish production code.
----------------------	---

This agent belongs in the agent plane of the platform and should reuse the same permission registry, endpoint registry, and audit trail used elsewhere.

11. Data imports, staffing views, and operational screens

Several of the reviewed documents are highly specific screen specs. Their value is that they reveal the intended operating model in concrete terms. They should be treated as sub-specs beneath the master architecture rather than independent projects.

Sub-spec	Why it matters to the master plan
Dashboard	Defines the supervisor command center and clarifies KPI priorities, urgency, and next-action routing.
Tasks	Defines the operational work queue and reinforces queue-first, inline-action UX.
Inspections	Defines quality-control flow and follow-up task generation from failed or noted inspections.
Issues	Defines housekeeping-visible defects and the bridge into maintenance escalation.
Imports	Confirms that spreadsheet-to-system synchronization is a core operational capability, not an afterthought.
Staff Shifts	Connects Workforce scheduling data to live housekeeping dispatch and workload balancing.

11.1 Imports strategy

The imports module should validate, preview, and execute repeatable CSV/XLSX imports for rooms, live status, tasks, inspections, issues, inventory pars, and staff shifts.

Spreadsheet workflows are a feature, not a temporary hack. The system should let staff continue using spreadsheets operationally while Workforce remains the canonical backend.

11.2 Staffing and labor visibility

- HKOps Staff Shifts is not a schedule editor or payroll tool.
- It is an operational labor view that joins live work, room assignments, unassigned pressure, and sector coverage back to Workforce schedule and timeclock data.
- This is the bridge between workforce capacity and room-readiness execution.

12. Delivery roadmap and first PR slices

The archive contains several phase plans. After normalization, the delivery sequence below is the most coherent and lowest-risk path.

Phase	Primary goals	Typical deliverables
Phase 1	Foundation and schema alignment	Tenancy-safe HKOps models, property hierarchy tables, Silver Sands seed command, basic room/task/summary APIs, shared web shell groundwork
Phase 2	Operational room workflows	Room board, bulk room status actions, task queue, dashboard summary, issue and inspection data paths
Phase 3	Admin console maturation	Superadmin and Tenant Admin route trees, permission-driven nav, business/location context selectors, control-plane data pages
Phase 4	Imports, staffing, and quality	Imports/data sync screen, Staff Shifts view, fuller inspection and issue workflows, supply/asset tabs
Phase 5	Event foundation	event_store migration, event service, timeline endpoint, initial event wiring across tasks/issues/shifts/CRM
Phase 6	Agent and dashboard extensibility	Widget manifests, preview/publish flow, installed widgets, domain packs
Phase 7	Scale and externalization	projection workers, optional broker integration, outbound publishers, richer read models and analytics

12.1 Best first integrated PR

- Add normalized property hierarchy models and migrations.
- Extend existing HKOps room models with Silver Sands–relevant metadata and hierarchy references.
- Add room asset and supply par entities plus seed data.
- Expose room list, room detail, room summary, and bulk room status APIs.
- Seed Silver Sands building/floor/sector/room structure.
- Refactor the existing web shell enough to support /hospitable routes and context-aware navigation.

12.2 Best second integrated PR

- Task queue and issue/inspection data endpoints.
- Supervisor dashboard and room board UI.
- Business and location selectors wired to live context.
- Event-store foundation for task and issue transitions.

13. Guardrails, risks, and normalization decisions

13.1 Hard guardrails

- Do not create a parallel housekeeping platform when HKOps already exists in the repository family.
- Do not use browser localStorage as production state for property operations.
- Do not use job titles in authorization logic.
- Do not skip business_id and location_id enforcement in query paths.
- Do not start the event layer with broker complexity when a transactional Postgres-first design is sufficient.
- Do not hard-code Silver Sands assumptions into shared UI or shared backend flows.

13.2 Key risks to manage

- Repository-path drift across documents may tempt duplicate modules or duplicate APIs; align with the existing implementation patterns before adding new code.
- UI shell refactors can accidentally break existing Hospitable flows; preserve functionality and re-home routes incrementally.
- Event payloads can sprawl if there is no registry and validation discipline.
- Imports can become dangerous if validation and preview are skipped; enforce row-level error transparency and repeatable job history.
- Tenant isolation bugs are especially likely in admin and timeline queries; prioritize tests for cross-tenant and cross-location denial cases.

13.3 Explicit reconciliations

Tension in archive	Resolved direction
Standalone Hospitable app vs integrated module	Integrated module inside Workforce, with clear module boundaries.
Kafka-first projection note vs Postgres-first build spec	Postgres-first event store and outbox-compatible design in Phase 1; broker later if justified.
Hospitable-only frontend vs broader admin console	One shared Workforce console with Hospitable as a workspace.
Simple motel blueprint vs enterprise platform scope	Use Silver Sands as the concrete seed case while keeping all models tenancy-safe and future-proof.

14. Recommended immediate deliverables

The archive is now strong enough to support a stable master plan. The best immediate outputs are implementation-ready and reviewable rather than more exploratory planning.

- Adopt this consolidated plan as the top-level master document for current direction.
- Create one source-of-truth architecture file in the repo and relegate prior drafts to archive/reference status.
- Build the Silver Sands seed path and room board APIs first so the platform produces visible operational value quickly.
- Start the event layer with a small, disciplined canonical event set rather than attempting universal coverage on day one.
- Treat the AI widget agent as a controlled extensibility layer that lands only after manifest validation and dashboard persistence are in place.

Recommended master-plan sentence for repo use

Workforce is the multi-tenant system of record for business, location, user, schedule, and permission context. Hospitable is the integrated property-operations workspace for room lifecycle, housekeeping execution, inspections, issues, assets, and supply standards, first validated through the Silver Sands Motel seed implementation and supported by a transactional event layer and a unified Workforce web console.

Appendix A. Core implementation checklist

- Normalize and register property, room asset, and supply models.
- Add Alembic migration with safe compatibility for current HKOps tables.
- Implement Silver Sands seed command.
- Deliver room board summary and bulk status APIs.
- Refactor the web shell and re-home Hospitable routes under /hospitable.
- Wire business/location context selectors and permission-aware nav filtering.
- Add event_store and event service skeleton with task and issue events.
- Create tests for tenant isolation, role enforcement, and Silver Sands seed idempotency.